Carpool Marketplace Payment Feature Design Document

Payments, Cards, Stripe Connect, Payouts, Ledger, Reconciliation, Disputes, Wireframes and Database Design

Version 1.0 - Development Ready Design

Document Overview

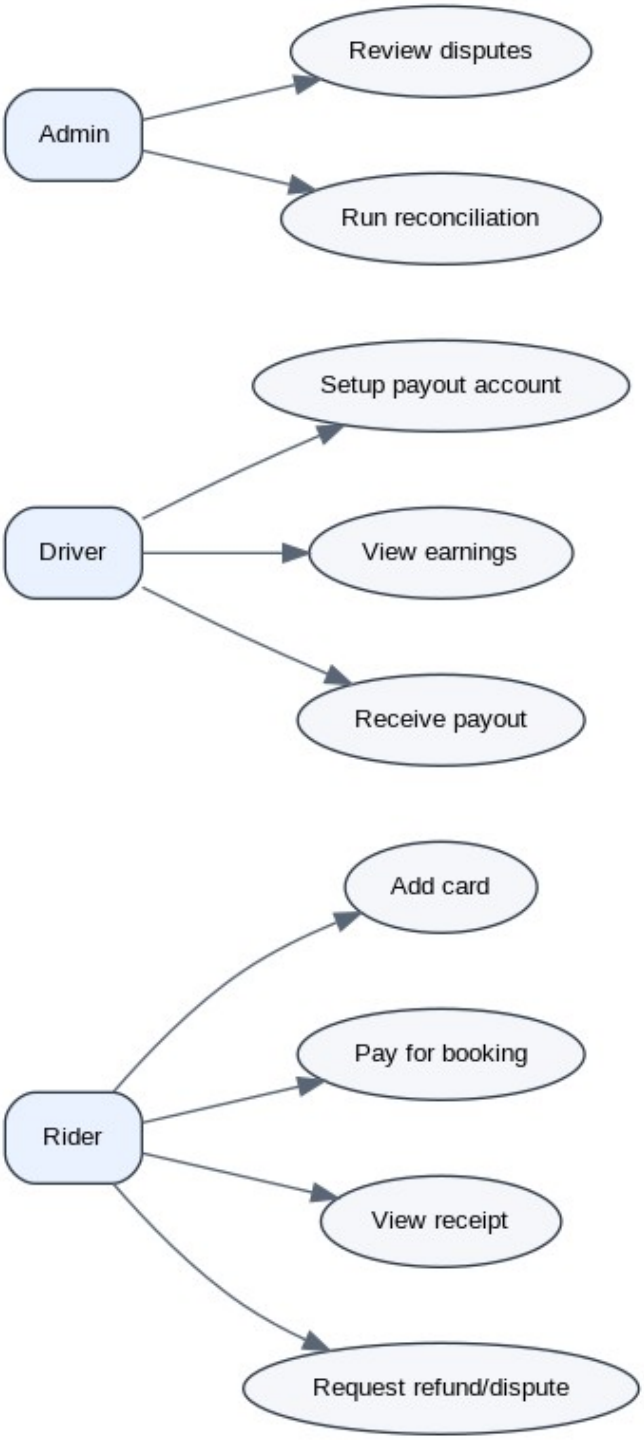
This document defines the complete payment feature for the carpool marketplace. It assumes ride publishing, seat booking, ride start, OTP pickup, drop-off and dispute signals already exist. The payment system consumes those signals to collect money from riders, hold funds safely, release payouts to drivers, process refunds, reconcile with Stripe and give admins operational control.

- Audience: product, frontend, backend, QA, DevOps and operations teams.
- Scope: payment methods, checkout, Stripe Connect onboarding, driver earnings, transfers, refunds, disputes, ledger, reconciliation, webhooks, queues and wireframes.
- Out of scope for MVP: stored-value wallet, cash payments, crypto, manual bank transfers, complex tax reporting and cross-border multi-currency settlement beyond payment provider support.

1. Product Goals and Decisions

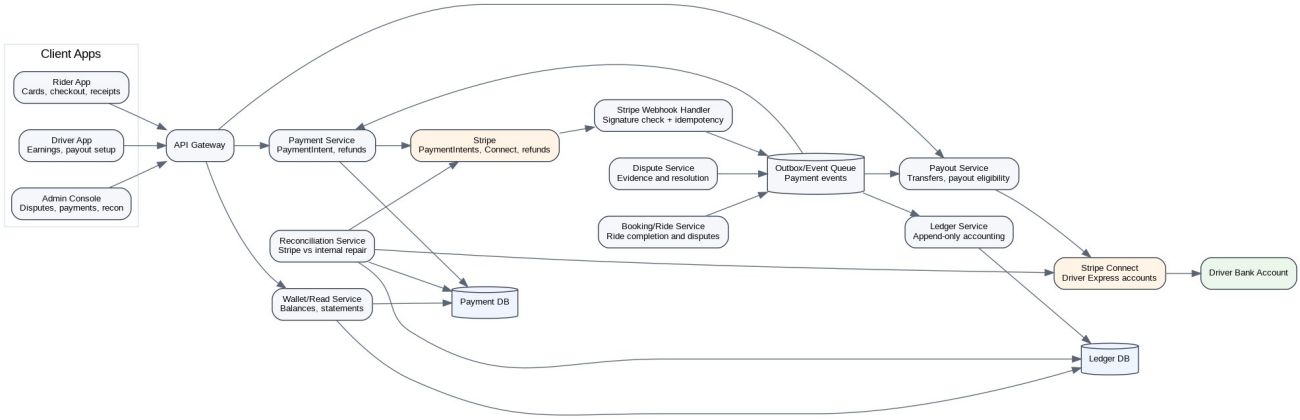
Area	Decision	Reason
Payment provider	Stripe + Stripe Connect Express	Supports marketplace payments, hosted driver onboarding and card checkout.
Rider payment timing	Pay at booking confirmation	Reduces no-shows and confirms seat commitment.
Money holding model	Escrow-like internal hold	Payment is captured, but driver payout waits for ride outcome and dispute window.
Driver payout	Bank payout through Stripe connected account	Avoids cash/manual settlement and keeps marketplace compliant with provider flow.
Accounting	Append-only ledger	Reliable audit trail and reconciliation.
Async processing	Outbox/event queue	Payment state should not depend only on synchronous API responses.
Reconciliation	Dedicated reconciliation service	Repairs mismatch between Stripe and internal system.

2. User Roles and Use Cases



Actor	Use cases
Rider	Add card, pay for booking, view receipt, see payment status, raise dispute or refund request.
Driver	Setup payout account, complete Stripe onboarding, view earnings, see pending/available balances, receive payouts.
Admin	Review disputes, inspect ledger, retry failed transfers, run reconciliation, refund riders, release or split payment.
System	Consume booking/ride events, process Stripe webhooks, calculate payout eligibility, reconcile mismatches.

3. High Level Architecture



The payment architecture is event-driven. Synchronous APIs create payment or onboarding sessions, but all final financial state transitions are confirmed using webhooks and internal events. This prevents data loss during server failures, duplicate webhooks and provider delays.

Component	Responsibility
Payment Service	Creates PaymentIntents, handles payment records, refunds and payment status.
Stripe Webhook Handler	Validates webhook signature, stores raw event, deduplicates by event ID and publishes internal events.
Outbox/Event Queue	Reliable internal event transport for payment, ledger, payout and dispute events.
Payment Processor	Applies payment events idempotently and coordinates booking, ledger and payout transitions.
Ledger Service	Writes immutable accounting entries and exposes balances/statements.
Payout Service	Calculates eligible payout, creates Stripe transfers, handles payout failure/retry.
Reconciliation Service	Compares Stripe objects with internal state and auto-repairs safe mismatches.
Wallet/Read Service	Presents rider transactions and driver earnings screens.
Dispute Service	Freezes disputed booking payments and sends resolution decisions to Payment Service.

4. Rider Payment and Card Flow

4.1 Rider Checkout Flow

Step	Frontend	Backend	Stripe
1	Rider opens booking checkout screen.	Validates booking is still payable and amount has not changed.	No call.
2	Rider taps Pay.	Creates PaymentIntent with amount, currency, metadata bookingId, riderId, rideId.	Creates PaymentIntent and returns client_secret.
3	App confirms card payment using Stripe SDK.	Waits for webhook, optionally returns pending state.	Processes card authorization/capture.
4	Shows success or failure.	Webhook marks payment PAID and booking payment complete.	Sends payment_intent.succeeded or failed.

4.2 Rider Payment Screens

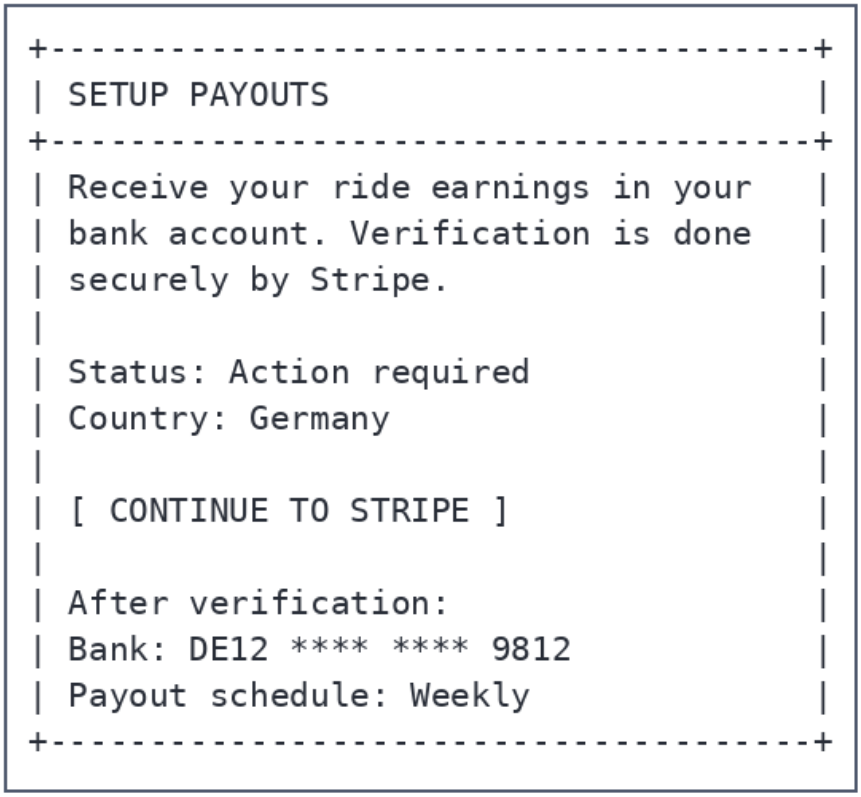
```
+--------------------------------------+
| PAY FOR YOUR RIDE                    |
+--------------------------------------+
| Hamburg -> Tallinn                   |
| Pickup: Stop 2   Drop: Stop 4        |
|                                      |
| Fare                          EUR 20 |
| Platform fee                  EUR  2 |
| Total                         EUR 22 |
|                                      |
| Payment method                       |
| Visa **** 3421              Change   |
|                                      |
| [ PAY EUR 22 ]                       |
+--------------------------------------+
```

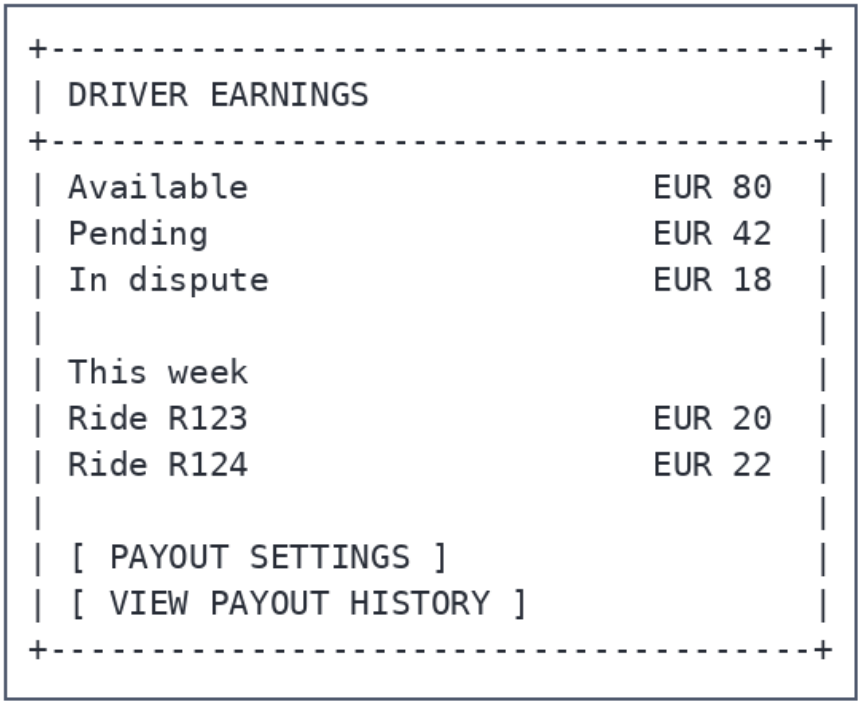
```
+--------------------------------------+
| PAYMENT METHODS                      |
+--------------------------------------+
| Saved cards                          |
|                                      |
| Visa **** 3421              Default  |
| Mastercard **** 8721                 |
|                                      |
| [ ADD CARD ]                         |
|                                      |
| Recent transactions                  |
| Hamburg -> Tallinn        EUR 22     |
| Status: Held until ride completion   |
+--------------------------------------+
```

5. Driver Stripe Connect and Payout Setup

5.1 Connect Onboarding Flow

Drivers should not enter sensitive KYC or bank details directly into your app for MVP. The app creates a Stripe Connect Express account and redirects the driver to Stripe hosted onboarding. Store only the connected account ID and status flags, not bank details.

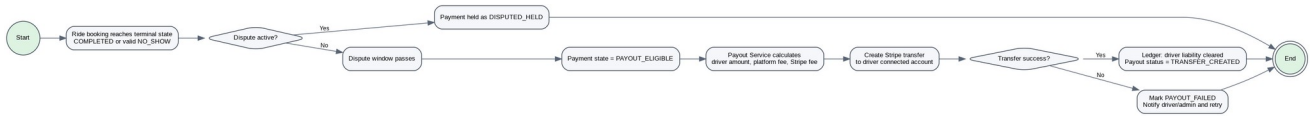




Connect state	Meaning	Driver UI
NOT_CREATED	Driver has no connected account.	Show Setup payouts.
ONBOARDING_STARTED	Stripe account exists but requirements are incomplete.	Show Continue verification.
ACTION_REQUIRED	Stripe requires additional information.	Show Fix payout setup.

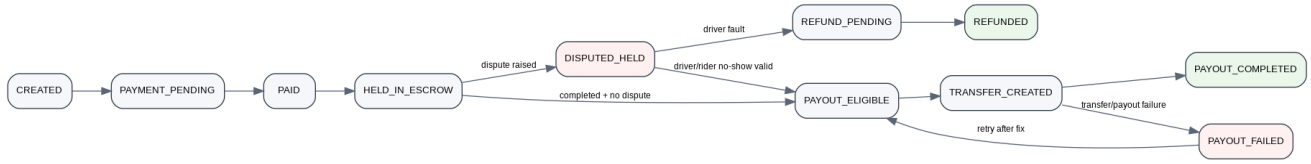
VERIFIED	charges_enabled and payouts_enabled are true.	Show Connected and bank summary.
RESTRICTED	Account disabled/restricted by provider.	Show Contact support / update details.

6. Payout Lifecycle



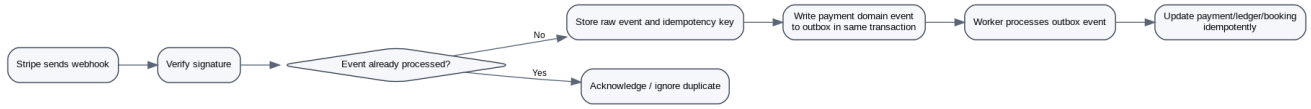
Trigger	Condition	Result
Booking completed	Rider dropped off and no active dispute.	Payment enters dispute window.
Dispute window passed	No dispute raised.	Booking payment becomes PAYOUT_ELIGIBLE.
Valid rider no-show	Driver waited and system evidence supports driver.	Driver payout eligible after dispute window.
Driver missed pickup	Evidence supports rider.	Refund rider, no driver payout for booking.
Transfer created	Driver Connect account verified.	Funds transferred to driver connected account.
Transfer failed	KYC/bank/provider failure.	Mark payout failed, notify driver/admin, retry after fix.

7. Payment State Machine



State	Description	Allowed next states
CREATED	Payment record exists before provider call.	PAYMENT_PENDING
PAYMENT_PENDING	PaymentIntent created or waiting for card completion.	PAID, PAYMENT_FAILED
PAID	Stripe confirms payment success.	HELD_IN_ESCROW
HELD_IN_ESCROW	Funds captured but not payable to driver yet.	PAYOUT_ELIGIBLE, DISPUTED_HELD, REFUND_PENDING
DISPUTED_HELD	Booking payment frozen because of dispute.	REFUND_PENDING, PAYOUT_ELIGIBLE, SPLIT_PENDING
PAYOUT_ELIGIBLE	Booking has passed completion and dispute checks.	TRANSFER_CREATED
TRANSFER_CREATED	Stripe transfer created for driver.	PAYOUT_COMPLETED, PAYOUT_FAILED
PAYOUT_COMPLETED	Driver settlement completed for this booking/payout batch.	Terminal
REFUNDED	Rider refund completed.	Terminal

8. Event Driven Design and Message Queue

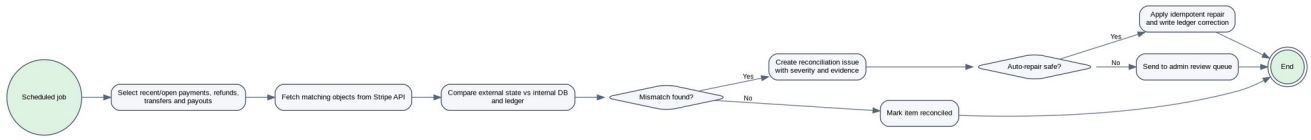


For MVP use the Postgres outbox pattern. It is simpler than Kafka and still reliable. Every domain update that must trigger payment work should also write an outbox row in the same database transaction. A worker processes unsent rows with retry and idempotency.

Event	Producer	Consumer	Purpose
PAYMENT_INTENT_CREATE	Payment Service	Ledger/Payment Processor	Track checkout initiation.

D			
PAYMENT_SUCCEEDED	Webhook Handler	Payment Processor, Ledger, Booking Service	Mark booking paid.
PAYMENT_FAILED	Webhook Handler	Payment Processor, Notification Service	Mark payment failed and notify rider.
BOOKING_COMPLETED	Booking Service	Payout Service	Start dispute-window/payout eligibility calculation.
DISPUTE_RAISED	Dispute Service	Payment Service	Freeze booking payment.
DISPUTE_RESOLVED_REFUND	Dispute Service	Payment Service, Ledger	Refund rider.
DISPUTE_RESOLVED_PAYOUT	Dispute Service	Payout Service	Release/continue payout.
PAYOUT_ELIGIBLE	Payout Service	Payout Worker	Create transfer.
TRANSFER_CREATED	Stripe/Webhook	Ledger, Wallet Service	Update payout state.
TRANSFER_FAILED	Stripe/Webhook	Payout Service, Notification	Retry or request driver action.
REFUND_SUCCEEDED	Stripe/Webhook	Ledger, Dispute Service	Close refund state.

9. Reconciliation Service



Reconciliation protects against missed webhooks, duplicate processing, provider delays and manual interventions. It should not replace webhooks; it is a repair and audit mechanism.

Mismatch	Example	Auto action
Stripe paid, internal pending	Stripe PaymentIntent succeeded but DB PAYMENT_PENDING.	Mark PAID, create ledger entries, mark booking paid.
Internal paid, Stripe failed	Internal PAID but Stripe PaymentIntent failed/canceled.	Open high severity reconciliation issue; do not auto payout.
Refund succeeded, internal pending	Stripe refund succeeded but DB REFUND_PENDING.	Mark REFUNDED and write reversal ledger entry.
Transfer failed, internal pending	Stripe transfer failed but DB TRANSFER_CREATED.	Mark PAYOUT_FAILED, notify driver/admin.
Ledger amount mismatch	Ledger fee split differs from Stripe amount.	Create issue, block payout if unresolved.

- Run hourly for recent and open payments from the last 48 hours.
- Run daily for all payments, refunds, transfers and payout batches from the previous settlement window.
- All repairs must be idempotent and written as ledger adjustments, never destructive edits.

10. Ledger Design

Use an append-only double-entry inspired ledger. Do not rely on mutable balance fields as source of truth. Balances shown on the UI are read models derived from ledger entries and payout states.

Ledger entry type	Debit/Credit meaning	Example
RIDER_PAYMENT_RECEIVED	Platform receives rider money.	Rider pays EUR 22.
DRIVER_EARNING_LIABILITY	Amount owed to driver.	Driver earning EUR 20.
PLATFORM_FEE_REVENUE	Platform fee portion.	Platform fee EUR 2.
STRIPE_FEE_EXPENSE	Payment processing fee.	Stripe fee EUR 0.80 if known.
REFUND_TO_RIDER	Refund reversal.	Refund EUR 22 to rider.
DRIVER_TRANSFER_CREATED	Driver liability reduced by transfer.	Transfer EUR 20.
PAYOUT_FAILURE_REVERSAL	Reopen liability if transfer fails.	Reverse failed payout.

Example calculation	Amount
Seat fare	EUR 20.00
Platform fee charged to rider	EUR 2.00
Rider total	EUR 22.00
Driver earning liability	EUR 20.00
Platform gross revenue	EUR 2.00
Payment provider fee	Stored when available from provider balance transaction

11. Database Model

payment_method

Column
id PK
user_id
stripe_customer_id
stripe_payment_method_id
brand
last4
exp_month
exp_year
is_default
status
created_at
updated_at

payment

Column
id PK
booking_id
ride_id
rider_id
stripe_payment_intent_id
amount_total
currency
fare_amount
platform_fee_amount
status
failure_reason
created_at
updated_at

stripe_connected_account

Column
id PK
driver_id
stripe_account_id
status
charges_enabled
payouts_enabled
requirements_due_json
created_at
updated_at

payout_batch

Column
id PK
driver_id
status
currency
amount_total
stripe_transfer_id
stripe_payout_id
created_at
updated_at

payout_item

Column
id PK
payout_batch_id
booking_id
payment_id
driver_amount

platform_fee
status
created_at
updated_at

ledger_entry

Column
id PK
entry_group_id
payment_id
booking_id
user_id
account_type
entry_type
direction
amount
currency
metadata_json
created_at

payment_event_outbox

Column
id PK
event_type
aggregate_type
aggregate_id
payload_json
status
retry_count
next_retry_at
created_at
processed_at

stripe_webhook_event

Column
id PK
stripe_event_id UNIQUE
event_type
payload_json
processing_status
received_at
processed_at

reconciliation_issue

Column
id PK
object_type
object_id
severity
internal_state
provider_state
description
status
created_at
resolved_at

refund

Column
id PK
payment_id
booking_id
stripe_refund_id
amount
currency
reason

status
created_at
updated_at

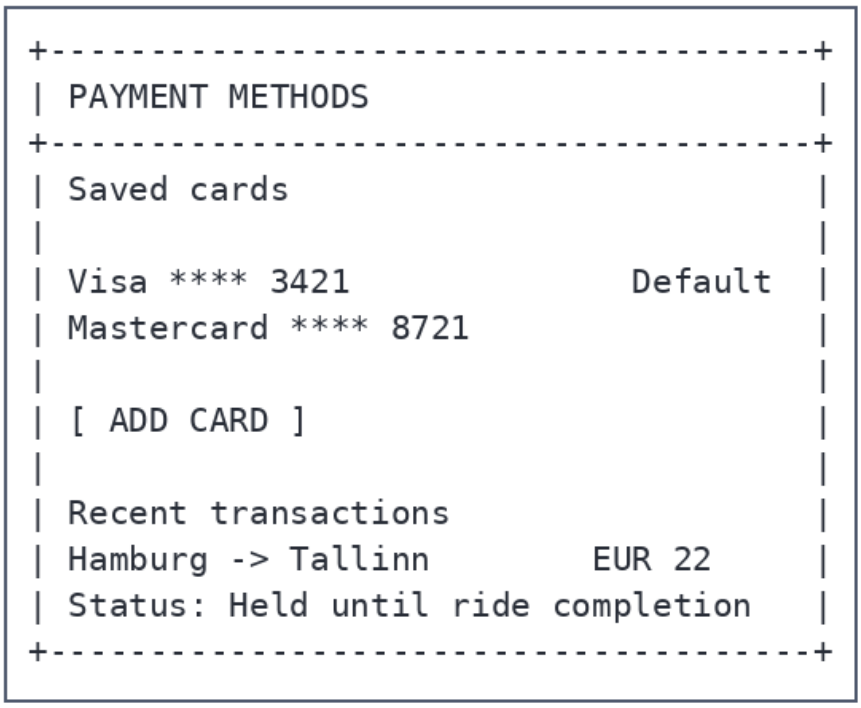
12. API Design

API	Method	Purpose	Notes
/payments/payment-intents	POST	Create checkout PaymentIntent.	Requires bookingId and amount validation.
/payments/{paymentId}	GET	Get payment status.	Used by rider payment details screen.
/payment-methods	GET	List saved cards.	Cards are tokenized by Stripe.
/payment-methods/setup-intent	POST	Create setup intent for saving a card.	Use Stripe SDK client secret.
/payment-methods/{id}/default	POST	Set default card.	Only stores provider token reference.
/drivers/me/connect-account	POST	Create/fetch driver Connect account.	Returns onboarding or dashboard link.
/drivers/me/earnings	GET	Driver earnings summary.	Available, pending, dispute and paid out.
/drivers/me/payouts	GET	Payout history.	Backed by payout_batch and ledger.
/webhooks/stripe	POST	Receive Stripe webhook.	Verify signature, store raw event, deduplicate.
/admin/payments/disputes	GET	Payment dispute queue.	Filters by status, SLA, amount.
/admin/payments/disputes/{id}/resolve	POST	Resolve payment dispute.	Refund, payout, split, escalate.
/admin/reconciliation/issues	GET	List reconciliation issues.	Requires admin role.

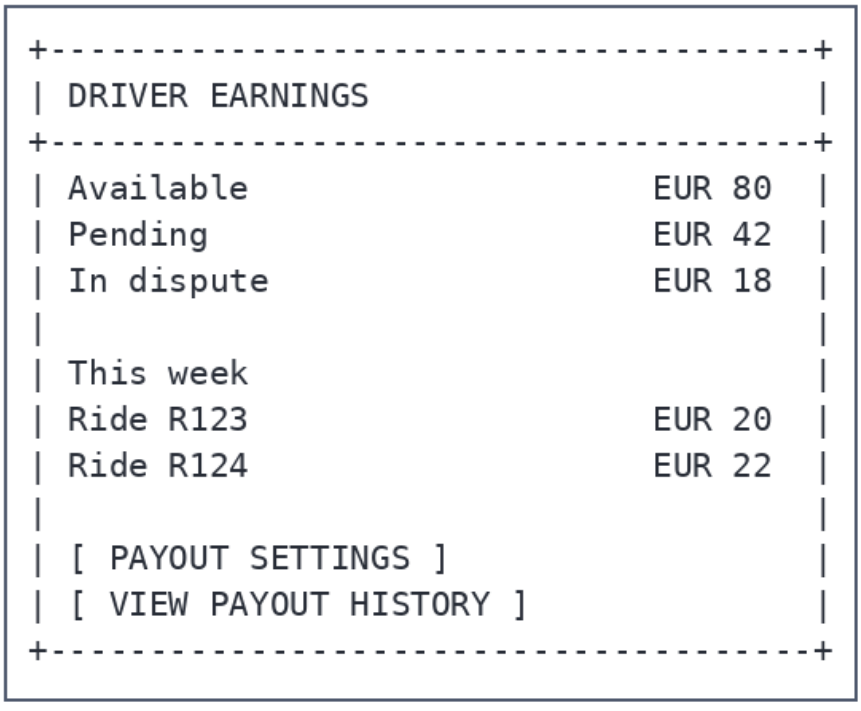
13. Wireframes

13.1 Rider checkout

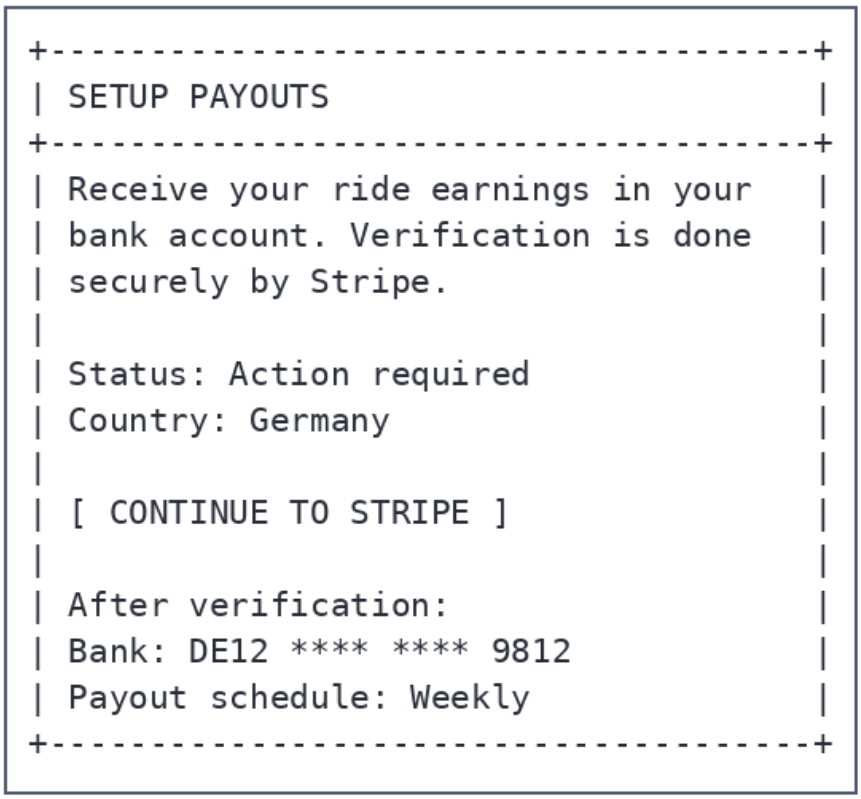
13.2 Rider cards and transactions



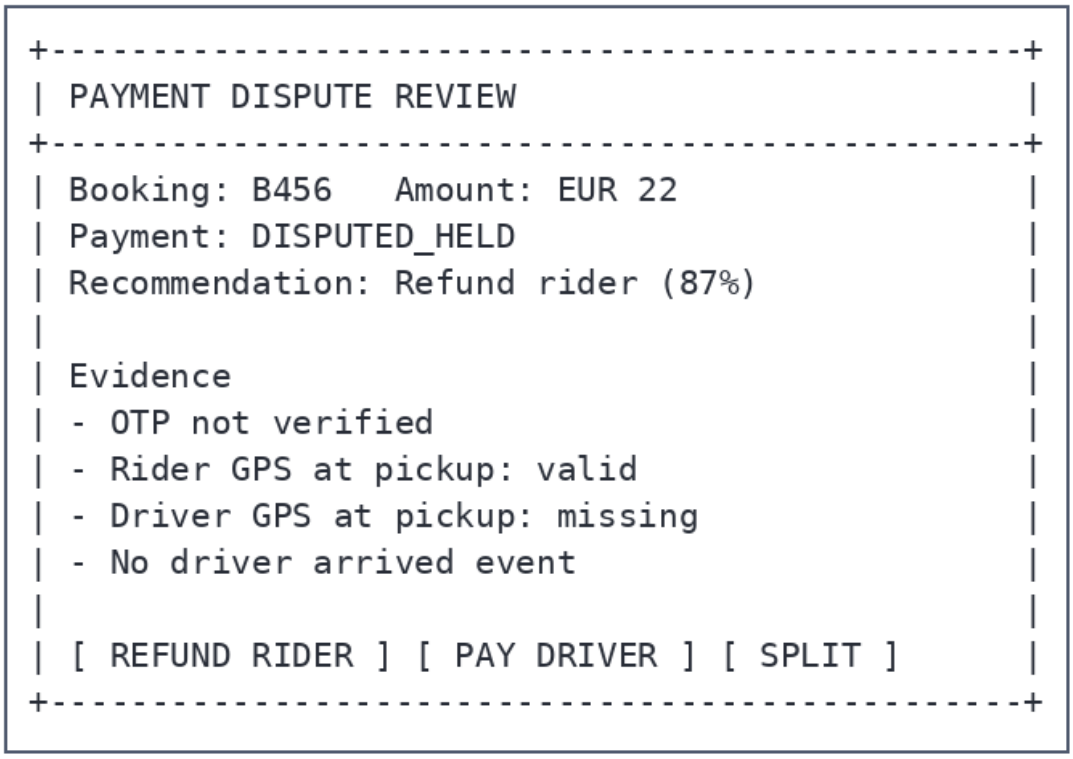
13.3 Driver earnings



13.4 Driver payout setup



13.5 Admin payment dispute



14. Disputes and Payment Freezing

Payments are frozen per booking, not per entire ride. If Rider 1 disputes, Rider 2 and Rider 3 can still become payout eligible if their bookings are clean.

Dispute type	Evidence	Payment decision
Rider no-show	Driver arrived event, GPS within pickup radius, wait timer, OTP missing, rider not near pickup.	Driver payout after dispute window if evidence supports driver.
Driver missed pickup	Rider GPS at pickup, driver GPS absent, no OTP, no arrived event, chat/call attempts.	Refund rider if evidence supports rider.
Wrong drop-off	Drop-off geofence failed, rider report, driver GPS trail.	Manual review; payment held.
Safety issue	Rider report, chat, support notes, emergency flag.	Escalate; payment held until admin decision.
Payment duplicate	Multiple PaymentIntents or duplicate charge indication.	Reconcile and refund duplicate if confirmed.

15. Edge Cases

Edge case	Expected behavior
Stripe webhook arrives twice	Use stripe_event_id uniqueness and idempotent processor; ignore duplicate.
Backend down during payment	Stripe still sends webhook later; reconciliation repairs if missed.
Payment succeeds but app times out	Rider sees pending; webhook marks paid; polling returns updated status.
Booking canceled after payment before ride start	Apply cancellation policy; full/partial refund via Payment Service.
Dispute raised after payout eligible but before transfer	Freeze payment and cancel pending payout item.
Dispute raised after driver transfer	Admin review may create platform loss or driver debit policy; avoid this by dispute window before transfer.
Driver Connect account not verified	Accumulate pending earnings; block transfer and show action required.
Transfer fails	Mark PAYOUT_FAILED, notify driver, retry after bank/KYC is fixed.
Refund fails	Keep REFUND_PENDING, retry, alert admin.
Currency mismatch	Reject if ride currency and payment currency differ unless explicit FX handling exists.
Ledger mismatch	Block payout and create reconciliation issue.
Multiple riders in same ride	Process payment and payout per booking item, not per ride.

16. Security, Compliance and Reliability

- Never store raw card numbers, CVV or bank account data in your system.
- Use Stripe SDK/Elements for card entry and Stripe hosted onboarding for driver payout data.
- Verify webhook signatures before storing and processing webhook payloads.
- Use idempotency keys for PaymentIntent creation, refunds and transfers.
- Restrict admin payment actions with role-based access and audit logs.
- Encrypt sensitive metadata at rest where needed and avoid storing unnecessary PII.
- Log all payment decisions, but do not log card details or secret keys.
- Use separate Stripe test and live credentials and isolate environments.

17. MVP Implementation Plan

Phase	Features
Phase 1 - Checkout	Create PaymentIntent, save card, mark booking paid from webhook, rider receipts.
Phase 2 - Driver payouts	Connect Express onboarding, earnings screen, payout eligibility after ride completion.
Phase 3 - Ledger/outbox	Append-only ledger, outbox processing, idempotent webhook handler.

Phase 4 - Disputes	Freeze booking payment, admin dispute screen, refund/payout decisions.
Phase 5 - Reconciliation	Hourly and daily jobs, repair mismatches, admin reconciliation issues.

18. Acceptance Criteria

- Rider can add a card and pay for a booking.
- Payment state is updated only after trusted Stripe webhook or reconciliation repair.
- Driver can complete Stripe Connect onboarding and see payout readiness.
- Booking completion creates payout eligibility only after dispute rules pass.
- Dispute freezes only the disputed booking payment.
- Ledger entries exist for every payment, refund and transfer.
- Duplicate webhook/event processing does not duplicate ledger or payouts.
- Reconciliation detects and repairs safe mismatches.
- Admin can view and resolve payment disputes with evidence.

19. Recommended Development Prompt

Implement the carpool marketplace payment feature using Stripe Connect Express. Create rider checkout, saved cards, driver payout setup, earnings, admin dispute review, ledger, outbox events, webhooks and reconciliation. Payment must be per booking. Use an append-only ledger. Webhooks and reconciliation must be idempotent. Payments must be held until ride outcome and dispute window. Payouts must go to driver Stripe connected accounts. Include database migrations, APIs, service classes, unit tests and integration tests for duplicate webhook, failed transfer, refund and dispute scenarios.